

Dossier Professionnel

Candidat : Vladimir Rodzaevskiy **Date de naissance** : 31 mai 1982 **Titre Professionnel visé** : Développeur Web et Web Mobile (DWWM) **Niveau** : 5 (équivalent Bac +2) **Code RNCP** : 37674 **Centre de formation** : AFPA Marseille **Année** : 2026

Sommaire

1. **Présentation du parcours**
2. **Activité-Type n°1 — Développer la partie front-end** - Exemple 1.1 : Authentification & session avec contextes React - Exemple 1.2 : Grille de planning hebdomadaire avec drag-and-drop natif - Exemple 1.3 : Modale Smart Planner — densité prévue par service
3. **Activité-Type n°2 — Développer la partie back-end** - Exemple 2.1 : Solver Smart Planner sous contraintes Convention HCR - Exemple 2.2 : Authentification JWT + bcrypt + middlewares composables - Exemple 2.3 : Génération de flux iCalendar (RFC 5545) des shifts
4. **Projet professionnel**
5. **Annexes**

1. Présentation du parcours

1.1 Parcours académique

Année	Diplôme	Pays
2004	Master 2 Économie et Gestion d'Entreprise	Russie — Moscou
2012	Master 2 Commerce International — INSEEC	France — Paris
2025–2026	Titre Professionnel DWWM (en cours)	France — AFPA Marseille

1.2 Parcours professionnel

Période	Poste	Secteur
~10 ans	Manager + serveur de salle	Restauration
3 ans	Croupier	Casino
2025–2026	Reconversion en cours	Développement web

1.3 Motivations de la reconversion

Après une longue expérience dans les métiers de service à la personne (restauration et casino), j'ai souhaité me réorienter vers un métier qui combine plusieurs aspects qui me sont importants :

- **Réflexion structurée** — la résolution de problèmes algorithmiques s'apparente à la logique économique que j'ai étudiée en Master.
- **Création concrète** — voir un produit qu'on a construit utilisé par d'autres procure une satisfaction comparable à un service salle réussi.
- **Apprentissage continu** — l'écosystème du développement évolue rapidement, ce qui correspond à mon goût pour l'autoformation.
- **Conditions de travail** — souhait d'un métier moins physique et permettant plus de flexibilité horaire.

Mes deux Masters en économie m'ont donné une rigueur analytique que je retrouve dans la conception de logiciels (modélisation, abstraction, mesure des compromis). Mes années en salle de restaurant et en casino m'ont appris la **gestion sous pression**, la **rigueur des process** et la **communication claire** avec des publics variés — autant de qualités transposables dans une équipe de développement.

1.4 Compétences transversales acquises hors développement

Domaine	Apport pour le développement
Gestion d'équipe (manager)	Capacité à coordonner, déléguer, rendre compte
Service client (serveur)	Empathie utilisateur, sens du détail UX
Croupier (jeu, calculs rapides)	Rigueur, gestion du stress, attention soutenue
Master Économie et Gestion d'Entreprise (Moscou 2004)	Modélisation, raisonnement analytique, lecture critique
Master 2 Commerce International — INSEEC Paris (2012)	Compréhension des enjeux produit / marché, intégration culturelle

2. Activité-Type n°1 — Développer la partie front-end d'une application web

Description de l'activité-type

(Référentiel REAC DWWM — RNCP 37674)

Le développeur web et web mobile maquette et développe l'interface utilisateur d'une application web ou web mobile en utilisant les langages, frameworks et bibliothèques adaptés. Il intègre les recommandations de sécurité, d'accessibilité et d'éco-conception. Il vérifie le bon fonctionnement de l'interface développée.

Le projet support : Crew

Tous mes exemples portent sur **Crew**, une application web full-stack de **planification d'équipe** que j'ai développée en solo comme projet de fin de formation. Le manager configure pour chaque équipier son poste, son shift habituel et ses heures contractuelles ; le **Smart Planner** génère ensuite en un clic un planning de service hebdomadaire équitable, respectant la Convention collective HCR, et chaque équipier reçoit ses shifts dans le calendrier natif de son téléphone via un flux iCal. Démo : <https://crew-planner-hazel.vercel.app> — code : <https://github.com/Vladimir08880888/crew>

Compétences couvertes par mes exemples

- C1.1 — Maquetter une application
- C1.2 — Réaliser une interface utilisateur web statique et adaptable
- C1.3 — Développer une interface utilisateur web dynamique
- C1.4 — Réaliser une interface utilisateur avec une solution de gestion de contenu ou un framework

Exemple 1.1 — Authentification et session avec contextes React

Projet : Crew — planification d'équipe **Période** : Mai – Juillet 2026 **Stack** : React 18, React Router 6, Context API, fetch (AJAX)

Contexte

L'application est multi-utilisateurs et nécessite une gestion fine de la session : connexion, déconnexion, persistance du token JWT au reload, protection des routes privées, et accès à l'utilisateur courant depuis n'importe quel composant.

J'ai conçu un **système d'authentification basé sur des contextes React** (`AuthContext` pour la session, `FamilyContext` pour l'équipe active) et un composant `ProtectedRoute` qui redirige vers `/login` si l'utilisateur n'est pas connecté.

Réalisation

1. Stockage du token au login

```

// front/src/context/AuthContext.jsx (simplifié)
import { createContext, useContext, useState, useEffect } from 'react';
import { authApi } from '../api/auth.api';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // Récupération du user au reload si token présent
  useEffect(() => {
    const token = localStorage.getItem('crew_token');
    if (!token) { setLoading(false); return; }
    authApi.me()
      .then(setUser)
      .catch(() => localStorage.removeItem('crew_token'))
      .finally(() => setLoading(false));
  }, []);

  async function login(email, password) {
    const { token, user } = await authApi.login({ email, password });
    localStorage.setItem('crew_token', token);
    setUser(user);
  }

  function logout() {
    localStorage.removeItem('crew_token');
    setUser(null);
  }

  return (
    <AuthContext.Provider value={{ user, login, logout, loading }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);

```

2. Protection des routes

```
// front/src/components/layout/ProtectedRoute.jsx
import { Navigate } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';

export function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <div>Chargement...</div>;
  if (!user) return <Navigate to="/login" replace />;
  return children;
}
```

3. Câblage dans App.jsx

```
<Routes>
  <Route path="/login" element={<Login />} />
  <Route path="/dashboard" element={
    <ProtectedRoute><Dashboard /></ProtectedRoute>
  } />
  { /* ... */ }
</Routes>
```

Compétences mises en œuvre

- Gestion d'état global avec **React Context API**
- Effet de bord contrôlé (`useEffect`) pour réhydrater la session
- Routage déclaratif avec **React Router 6**
- Persistance via `localStorage` avec gestion des erreurs (token expiré → nettoyage automatique)
- Pattern « hook personnalisé » (`useAuth`) pour exposer l'API

Résultat

L'authentification est totalement transparente pour l'utilisateur : il ferme l'onglet, revient le lendemain, il est toujours connecté tant que le JWT (durée 7 jours) reste valide. Les routes privées sont protégées sans condition de race ni flash de contenu non autorisé.

Référence dans le code source

- `front/src/context/AuthContext.jsx`
- `front/src/components/layout/ProtectedRoute.jsx`
- `front/src/App.jsx`

Exemple 1.2 — Grille de planning hebdomadaire avec drag-and-drop natif

Projet : Crew — planification d'équipe **Période** : Mai – Juillet 2026 **Stack** : React 18, HTML5 Drag-and-Drop API native, CSS pur

Contexte

La page **Planning** est le cœur d'utilisation quotidien du manager. Il s'agit d'une grille **équipers × jours de la semaine** où chaque cellule représente les shifts d'un équipier sur une journée donnée. Le manager doit pouvoir réorganiser le planning à la souris : déplacer un shift d'un jour à l'autre ou d'un équipier à l'autre.

J'ai implémenté ce drag-and-drop avec l'**API HTML5 native**, sans bibliothèque externe, afin d'économiser ~30 KB de bundle et de garder le contrôle total du comportement.

Réalisation

1. Grille performante et responsive

- Rendu d'une grille HTML `<table>` scrollable horizontalement sur mobile, en-tête sticky, et pastilles de **santé du service** (Saine / Tendue / Risque) en bas de chaque colonne jour.
- Regroupement visuel des équipiers **par poste** (cuisine puis salle), avec un en-tête de groupe et une ligne « Extras » affichant les déficits de couverture par service.

2. Drag-and-drop natif avec rendu optimiste + rollback

```
// front/src/pages/Planning.jsx (extrait)
async function handleDrop(e, targetDate, targetUserId) {
  e.preventDefault();
  const shiftId = e.dataTransfer.getData('shift-id');
  const prev = shifts;

  // Mise à jour optimiste : on déplace le shift localement tout de suite
  setShifts(shifts.map(s =>
    s.id === Number(shiftId)
      ? { ...s, date: targetDate, user_id: targetUserId }
      : s));

  try {
    await shiftsApi.update(shiftId, { date: targetDate, user_id: targetUserId });
  } catch (err) {
    setShifts(prev); // rollback si le back rejette
    toast.fromError(err); // ex. : « Plafond HCR 48 h/semaine dépassé »
  }
}
```

Si le back-end rejette le déplacement (par exemple parce qu'il violerait le plafond hebdomadaire HCR de 48 h), l'état est **automatiquement restauré** et un toast explique le motif.

3. Validation de la polyvalence côté client

Un slot refuse les drops dans une zone visuellement marquée « incompatible » (le poste cible n'est pas dans la matrice de compétences de l'équipier), avec une animation CSS de refus.

Compétences mises en œuvre

- **HTML5 Drag-and-Drop API native** (`draggable` , `ondragstart` , `ondrop` , `dataTransfer`) — zéro dépendance
- **Rendu optimiste** systématique avec rollback automatique en cas d'erreur API
- Mémoïsation des dérivations de l'état (groupes de postes, extras) via `useMemo`
- **Responsive** : grille scrollable horizontale + tap-targets ≥ 44 px sur mobile

Résultat

Le manager réorganise son planning à la souris de façon fluide ; toute violation des règles légales est bloquée côté serveur et expliquée à l'utilisateur sans jamais laisser la grille dans un état incohérent.

Référence dans le code source

- `front/src/pages/Planning.jsx`

Exemple 1.3 — Modale Smart Planner avec densité prévue par service

Projet : Crew — planification d'équipe **Période** : Mai – Juillet 2026 **Stack** : React 18, hooks (`useState`, `useEffect`, `useMemo`), `react-i18next`

Contexte

La modale **Smart Planner** est le composant le plus dense fonctionnellement de l'application : c'est là que le manager déclare la **densité prévue** de chaque service de la semaine (combien de monde il attend) et obtient en retour une proposition de planning calculée par le back-end sous contraintes Convention HCR.

Réalisation

1. Tableau interactif de densité (7 jours × 2 services)

- Chaque cellule présente un input numérique 30–200 % avec 3 boutons-presets (**Calme 0,5**, **Normal 1,0**, **Chargé 1,3**).
- Boutons « Tous → 0,5 / 1,0 / 1,3 » qui remplissent une colonne entière en un clic.

2. Re-fetch automatique de la proposition à chaque changement

```
// front/src/components/planning/SmartPlannerModal.jsx (extrait)
useEffect(() => {
  setLoading(true);
  shiftsApi.generatePlan({
    family_id: familyId, from, to,
    capacityByDateAndService: perCell,
  })
  .then(setData)
  .finally(() => setLoading(false));
}, [familyId, from, to, perCell]);
```

Le manager voit l'effet de ses réglages en **quasi temps réel**. La réconciliation React fait office de debounce implicite : des keystrokes rapides n'entraînent qu'un seul re-fetch par batch.

3. Affichage du résultat en trois blocs

- heures par équipier (vs cibles contractuelles),
- couverture par (service × poste) avec fractions et %,
- services non couverts avec **motif explicite**.

Un bouton d'application POST en masse les shifts proposés et ferme la modale avec un toast récapitulatif (« 26 shifts créés »).

Compétences mises en œuvre

- **Composants contrôlés** avec une source de vérité unique (`perCell`)
- **useEffect** avec dépendances ciblées pour le re-fetch live
- Gestion du `loading` state séparée pour ne pas bloquer l'UI
- **i18n** systématique (react-i18next) — aucun label en dur

Résultat

Le manager pilote la génération de son planning de façon visuelle et itérative, sans jamais quitter l'écran ni recharger la page. Le pont entre l'UI et le solver back-end est entièrement réactif.

Référence dans le code source

- `front/src/components/planning/SmartPlannerModal.jsx`

3. Activité-Type n°2 — Développer la partie back-end d'une application web

Description de l'activité-type

(Référentiel REAC DWWM — RNCP 37674)

Le développeur web et web mobile crée, modifie, fait évoluer une base de données, en développe les composants d'accès aux données. Il développe la partie back-end d'une application web ou web mobile en appliquant les recommandations de sécurité, d'éco-conception et de performance. Il s'assure du bon fonctionnement par des tests appropriés.

Compétences couvertes par mes exemples

- C2.1 — Créer une base de données
- C2.2 — Développer les composants d'accès aux données
- C2.3 — Développer la partie back-end d'une application web
- C2.4 — Élaborer et mettre en œuvre des composants dans une application de gestion de contenu ou e-commerce

Exemple 2.1 — Solver Smart Planner sous contraintes Convention HCR

Projet : Crew — planification d'équipe **Période** : Mai – Juillet 2026 **Stack** : Node.js 22, Express, mysql2, algorithme greedy avec scoring multi-critères

Contexte

Le cœur métier de Crew est un **solver de planning hebdomadaire** qui doit, à partir des caractéristiques de chaque équipier (profil, poste, polyvalence, heures cibles), proposer un planning :

- atteignant une couverture idéale par poste (cuisine, salle),
- respectant strictement la **Convention collective HCR** (48 h/sem, 11 h/jour cuisinier, 11 h 30 autres, 2 jours de repos hebdo) et le **Code du travail** L3121-20,
- minimisant la masse salariale prévisionnelle,
- préférant les spécialistes du poste primaire, n'engageant la polyvalence (multi-skill) qu'en sous-effectif.

Réalisation

1. Constantes légales partagées

```
// back/src/services/plannerSolver.js
const HCR_WEEKLY_MAX = 48;           // Code du travail L3121-20
const HCR_MIN_REST_DAYS = 2;        // Convention HCR
const HCR_DAILY_MAX_BY_POSTE = {    // Convention HCR par poste
  cuisine: 11, salle: 11.5, plonge: 11.5,
  bar: 11.5, administration: 10,
};
const COST_PENALTY_WEIGHT = 1.0;
```

2. Filtre de candidats — contraintes dures inviolables

```

const candidates = members
  .filter((m) => {
    if (!m.weekly_hours_target) return false;
    if (!canFill(m, poste)) return false; // multi-skill
    const akey = `${m.user_id}-${date}`;
    if (assignedByUserDay.get(akey)?.has(service)) return false;

    const shiftDur = SHIFT_DURATIONS[service];
    const wouldBeWeek = hours[m.user_id].planned + shiftDur;

    // HCR : plafond hebdomadaire absolu 48 h.
    if (wouldBeWeek > HCR_WEEKLY_MAX) return false;
    // HCR : plafond quotidien selon poste.
    const dayHours = [...(assignedByUserDay.get(akey) || [])]
      .reduce((sum, st) => sum + SHIFT_DURATIONS[st], 0);
    if (dayHours + shiftDur > hcrDailyCap(m.poste)) return false;
    // HCR : minimum 2 jours de repos hebdo.
    const wouldDays = new Set([...userDays, date]).size;
    if (weekDates.length - wouldDays < HCR_MIN_REST_DAYS) return false;
    // Junior seul interdit (règle métier).
    if (!seniorPresent && !isSenior(m)) return false;
    return true;
  })

```

3. Scoring multi-critères avec pénalité économique

```

.map((m) => {
  const deficit = m.weekly_hours_target - hours[m.user_id].planned;
  let score = deficit * 10; // priorité besoin
  if (m.shift_default === service) score += 5; // préférence shift
  if (m.poste === poste) score += 3; // poste primaire
  if (m.level === 'chef') score += 2; // expérience
  // Pénalité coût : à déficit égal, choisir le moins cher.
  score -= (shiftCost(m, service, cfg) / 100) * COST_PENALTY_WEIGHT;
  return { member: m, score };
})
.sort((a, b) => b.score - a.score);

```

Compétences mises en œuvre

- **Algorithmie** : design d'un solver greedy avec scoring composite (4 facteurs métier + 1 facteur économique).
- **Modélisation de contraintes légales** dans le code (Convention collective + Code du travail). Toutes les valeurs seuils citées dans `JUSTIFICATION_SCIENTIFIQUE.md` avec sources peer-reviewed.
- **Séparation des responsabilités** : 3 fonctions pures (`canFill`, `coefOf`, `rateOf`) testables isolément.

- **Calculs flottants stables** : taux horaires stockés en centimes (entiers SQL) pour éviter l'arithmétique IEEE 754.
- **Logique métier complexe** combinant 7 dimensions (profil, poste, polyvalence, heures, repos, coût, capacité prévue).

Résultat

29 scénarios automatisés vérifient le solver enrichi (bloc 10 du plan de tests) : compliance HCR, pénalité coût qui ne dégrade jamais la couverture, polyvalence qui n'écrase pas les spécialistes. Tous les seuils sont configurables côté UI par le manager via la page Configuration.

Référence dans le code source

- `back/src/services/plannerSolver.js`
- `back/src/controllers/shifts.controller.js`
- `back/src/models/shift.model.js`
- `back/src/validators/shift.validator.js`
- `back/migrations/012_labor_costs.sql`

Exemple 2.2 — Authentification JWT + bcrypt + middlewares composables

Projet : Crew — planification d'équipe **Période** : Mai – Juillet 2026 **Stack** : Node.js 22, Express 4, jsonwebtoken, bcrypt, mysql2

Contexte

L'application multi-utilisateurs nécessite un système d'authentification sécurisé et un mécanisme d'autorisation **composable** : selon la route, on peut avoir besoin d'exiger « connecté », « membre d'une équipe », « membre actif d'une équipe », ou « administrateur (manager) d'une équipe ».

Plutôt que de dupliquer ces vérifications dans chaque controller, j'ai conçu une **chaîne de middlewares** combinables, sans cookie de session pour éliminer la classe de vulnérabilités CSRF.

Réalisation

1. Hachage bcrypt à l'inscription

```
// back/src/services/password.service.js
import bcrypt from 'bcrypt';

const COST = 10; // ~100ms par hash sur CPU moderne

export async function hashPassword(plain) {
  return bcrypt.hash(plain, COST);
}

export async function comparePassword(plain, hash) {
  return bcrypt.compare(plain, hash);
}
```

2. Signature JWT au login

```
// back/src/services/jwt.service.js
import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';

export function signToken(userId) {
  return jwt.sign({ sub: userId }, env.jwt.secret, {
    expiresIn: env.jwt.expiresIn, // 7d
    algorithm: 'HS256',
  });
}

export function verifyToken(token) {
  return jwt.verify(token, env.jwt.secret);
}
```

3. Middleware d'authentification

```
// back/src/middleware/auth.js
import { verifyToken } from '../services/jwt.service.js';
import { unauthorized } from '../utils/httpError.js';

export function authRequired(req, res, next) {
  const auth = req.headers.authorization || '';
  const [scheme, token] = auth.split(' ');
  if (scheme !== 'Bearer' || !token) {
    throw unauthorized('Non authentifié');
  }
  try {
    const { sub } = verifyToken(token);
    req.user = { id: sub };
    next();
  } catch (err) {
    throw unauthorized('Token invalide ou expiré');
  }
}
```

4. Middlewares d'autorisation équipe (composables)

```
// back/src/middleware/familyAccess.js
export async function requireFamilyMember(req, res, next) {
  const familyId = Number(req.params.familyId);
  const member = await familyMemberModel.findByFamilyAndUser(familyId, req.user.id);
  if (!member) throw forbidden('Pas membre de cette équipe');
  req.familyMember = member;
  next();
}

export function requireAdmin(req, res, next) {
  if (!req.familyMember?.is_admin) throw forbidden('Action réservée aux managers');
  next();
}
```

Note d'implémentation : dans le schéma SQL, une équipe est portée par les tables `families` / `family_members` (héritées du squelette initial du projet). Le terme métier exposé à l'utilisateur est partout « équipe ».

5. Chaînage déclaratif dans les routes

```
// back/src/routes/families.routes.js
router.post('/:familyId/regenerate-code',
  asyncHandler(requireFamilyMember),
  requireAdmin,
  asyncHandler(familiesController.regenerateCode));
```

Compétences mises en œuvre

- **bcrypt** pour le hachage (coût 10, résistant aux GPU)
- **JWT HS256** signé avec secret stocké en `.env` (jamais en dur)
- Pattern **middlewares Express composables** (chaque middleware fait UNE chose)
- Gestion d'erreurs cohérente avec exceptions typées (`unauthorized`, `forbidden`)
- Récupération du contexte utilisateur depuis `req.user`

Résultat

Sécurité au niveau état de l'art pour ce type d'application. Le test d'intégration `tests/playwright/` couvre 11 scénarios d'authentification (token absent, invalide, expiré, mauvais mot de passe, etc.) — tous passent.

Référence dans le code source

- `back/src/middleware/auth.js`
- `back/src/middleware/familyAccess.js`
- `back/src/services/jwt.service.js`
- `back/src/services/password.service.js`

Exemple 2.3 — Génération de flux iCalendar (RFC 5545) des shifts

Projet : Crew — planification d'équipe **Période** : Mai – Juillet 2026 **Stack** : Node.js 22, Express, bibliothèque `ical-generator`

Contexte

Le point fort technique du projet : permettre à chaque équipier de recevoir ses **shifts** directement sur son téléphone (iPhone, Android) sans installer d'application, en s'appuyant sur le standard **iCalendar (RFC 5545)** que tous les calendriers natifs comprennent.

J'ai conçu un service de génération iCal et des endpoints exposant le planning de l'équipier : un flux global (tous ses shifts, toutes équipes confondues) et un sous-flux par équipe.

Réalisation

1. Service de génération (un VEVENT par shift)

```
// back/src/services/ical.service.js
import ical from 'ical-generator';

export function buildIcal({ ownerName, calendarName, calendarColor, shifts }) {
  const cal = ical({
    name: calendarName || `Crew ${ownerName}`,
    prodId: { company: 'Crew', product: 'Planner', language: 'FR' },
    timezone: 'Europe/Paris',
  });
  if (calendarColor) cal.x('X-APPLE-CALENDAR-COLOR', calendarColor);

  shifts.forEach((s) => {
    const emoji = POSTE_EMOJI[s.poste] || ' ';
    const event = cal.createEvent({
      id: `shift-${s.id}@crew-planner`,
      start: shiftStart(s), // s.date + s.start_time (ou défaut du shift_type)
      end: shiftEnd(s),
      summary: `${emoji} Service ${s.shift_type} - ${s.poste}`,
      description: `Poste : ${s.poste}\nShift : ${s.shift_type}`,
      categories: [{ name: 'Planning service' }],
    });
    // VALARM : notification système 2 h avant le service
    event.createAlarm({ type: 'display', trigger: 7200,
      description: `Service ${s.shift_type} dans 2 h` });
  });

  return cal.toString();
}
```

2. Routes (authentification par token URL)

```
// back/src/routes/calendar.routes.js
router.get('/:token', asyncHandler(calendarController.export));
router.get('/:token/perso.ics', asyncHandler(calendarController.export));
router.get('/:token/family/:familyId.ics', asyncHandler(calendarController.exportFamily));
```

3. Controller

```
// back/src/controllers/calendar.controller.js (extrait)
async function userFromToken(token) {
  const cleaned = token.replace(/\\.ics$/, '');
  const user = await userModel.findByCalendarToken(cleaned);
  if (!user) throw notFound('Calendrier introuvable');
  return user;
}

export const calendarController = {
  // Flux principal : tous les shifts à venir de l'utilisateur
  async export(req, res) {
    const user = await userFromToken(req.params.token);
    const shifts = await shiftModel.listUpcomingForUser(user.id, 100);
    const ics = buildIcal({ ownerName: `${user.first_name} ${user.last_name}`, shifts });
    res.set('Content-Type', 'text/calendar; charset=utf-8');
    res.set('Content-Disposition', `inline; filename="crew-${user.first_name}.ics"`);
    res.send(ics);
  },
  // Sous-flux : shifts d'une seule équipe (exportFamily, similaire avec filtre SQL)
};
```

Compétences mises en œuvre

- Étude et **application d'un standard** (RFC 5545 iCalendar)
- Intégration de **bibliothèque tierce** (ical-generator) via npm
- Conception d'une **authentification alternative** (token URL plutôt que JWT, car le calendrier externe ne peut pas envoyer de header Authorization)
- Configuration de **headers HTTP** appropriés (Content-Type , Content-Disposition)
- Gestion de la **timezone** (Europe/Paris, passage heure d'été automatique)

Résultat

Chaque équipier s'abonne depuis iPhone Calendar, Google Calendar ou Outlook et reçoit ses shifts avec une notification système **2 h avant** chaque service, sans aucune application Crew à installer. Testé sur iOS 17 et Android 14.

Référence dans le code source

- back/src/services/ical.service.js
- back/src/controllers/calendar.controller.js
- back/src/routes/calendar.routes.js

4. Projet professionnel

4.1 Vision à court terme (6-12 mois post-formation)

À l'issue de la formation DWWM, je souhaite :

- **Maintenir et faire évoluer** Crew en production (l'application est déjà déployée sur **Vercel + Fly.io** — <https://crew-planner-hazel.vercel.app>) : ajout de notifications email, internationalisation, CI/CD GitHub Actions
- **Continuer à publier** mes projets personnels sur GitHub pour étoffer mon portfolio

4.2 Vision à moyen terme (1-3 ans)

- Monter en compétence sur le **back-end pur** (Node.js avancé, PostgreSQL, Redis, Docker, CI/CD)
- Acquérir une expérience d'équipe (revue de code, méthodes Agile en équipe, gestion d'incidents)
- Envisager une **activité indépendante** complémentaire : développement de petites applications métier pour PME du secteur hôtellerie-restauration (que je connais particulièrement bien) — caisse enregistreuse, gestion de réservations, suivi de stock, etc.

4.3 Atouts pour cette reconversion

Mon parcours hors développement constitue un avantage différenciant :

Atout	Apport en développement
2 Masters (Moscou 2004 + France 2012)	Capacité d'analyse, recul stratégique, autoformation
10 ans en restauration (manager)	Gestion d'équipe, sens du service, gestion du stress
3 ans en casino (croupier)	Rigueur des process, attention soutenue, calculs rapides
Maturité (44 ans)	Recul, stabilité, capacité à prioriser et tenir des délais
Reconversion choisie	Motivation intrinsèque forte (pas un choix par défaut)

Je vise des structures qui valorisent l'autonomie, la rigueur et la relation client — des qualités que mes années de service salle et casino ont profondément ancrées.

5. Annexes

Annexe	Référence
Code source Crew	https://github.com/Vladimir08880888/crew
Dossier de Projet détaillé	../dossier-projet/cahier-des-charges.md
Justification scientifique (12 réfs peer-reviewed)	../dossier-projet/annexes/JUSTIFICATION_SCIENTIFIQUE.md
Plan de tests Playwright (60+ scénarios)	../dossier-projet/annexes/PLAN_DE_TESTS.md

Annexe	Référence
Schéma de la base de données (Mermaid)	<code>../dossier-projet/annexes/SCHEMA_BDD.md</code>
Screenshots de l'application	https://github.com/Vladimir08880888/crew/tree/main/annexes/screenshots

Document rédigé en mai 2026 pour la soutenance du Titre Professionnel Développeur Web et Web Mobile à l'AFPA Marseille.